

Learning Objectives

This lab lets you practice reading from and writing to text files.

Estimated Size

1 program file: `files.py`, 2 functions.

Setup

In your Python files, you must include author information. On the first line, add a `# Author` comment line, where you mention your full name. Similarly, include `your email and Spire ID` on the `# Email` and `# Spire ID` comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author    : John Snow
# Email     : jsnow@umass.edu
# Spire ID  : 12345678
```

There is NO starter code for this lab. Instead, you will create a file named `files.py` from scratch. This is the only file you need to work on and submit.

1. Implement function `print_stars_to_file` (8 points)

Write a function called `print_stars_to_file`. It should take an integer `n` as a parameter (you can assume `n >= 1`), and it should create a text file named `stars_n.txt`, where `n` is the actual value of the parameter `n`. In this text file, it should contain `n` lines of text:

- the first line is `(n-1)` leading space characters followed by `1` star character `*`
- the second line is `(n-2)` spaces followed by `3` stars
- the third line `(n-3)` spaces followed by `5` stars
- the fourth line is `(n-4)` spaces followed by `7` stars
-
- the `n-th` line (last line) has `(2*n-1)` stars with no leading space character

For example, calling `print_stars_to_file(3)` would create a file named `stars_3.txt` which has the following content:

```
*
**
***
```

And calling `print_stars_to_file(6)` would create a file named `stars_6.txt` which has:

```
*
**
***
****
*****
*****
```

Specifically, your function should:

1. Open a file named `stars_n.txt` (where `n` is the value of the parameter) in `'w'` (write-only) mode.
2. Write `n` lines of stars to the file as described on the previous page. The number of leading space characters and stars must match the description exactly. There should be NO trailing space characters at the end of each line.
3. It's ok if your file contains a trailing empty line at the end, as long as that line has no character.
4. Do **NOT** ask the user for any input.
5. Do **NOT** include the function call to this function when you submit.

 **Tip:** There are several ways to write strings to a file. You can use a file's `write` function -- if you do, note that this function does not automatically write a newline character, so you need to include an explicit `'\n'` (i.e., newline) character at the end of each line. Alternatively, you can use the familiar `print` function, with an extra argument that says `file=f`, where `f` is the file variable. For example, `print('Hello', file=f)` will write the string `'Hello'` to the file you opened as variable `f`. The `print` function automatically adds a newline character each time you call it.

When you are done writing this function, call `print_stars_to_file(3)` in your program, and then check if it successfully created a file named `stars_3.txt` in the **current working directory** -- this is usually the directory you see at the Terminal window in VSCode. If you don't know how to find this directory, ask a staff member to help you. Open this file to see if it has the expected content. Feel free to try a few other parameter values to verify that all resulting files are correct.

 **Tip:** If you encounter any problem, we strongly encourage you to **use the debugger** to execute your code one line at a time and figure out what may be wrong. This is the standard way we debug and fix issues in a program. When you write a complex program, you can't expect to simply eyeball the code to figure out the problems -- the debugger is your best and most effective tool.

2. Implement function `calc_avg_from_file` (7 points)

Write a function called `calc_avg_from_file`. It takes no parameters, and its task is to calculate and return the average grade from a text file named `grades.txt`, which stores the students' grades, one number per line. For example, `grades.txt` may have the following content:

```
82.5
93
77.5
65
```

Your function should read from this file, get all the numbers, calculate, and return the average of them, which is `79.5` in this particular example. Specifically, your function should:

1. Open the file named `grades.txt` in `'r'` (read-only) mode.
2. Read the entire content of the file into a string. If you opened the file as `f`, you can do `text=f.read()` to read the whole content into a string called `text`. Using the above example, this string will have the following characters: `'82.5\n93\n77.5\n65'`. Note there is a newline character `'\n'` after each number (except the last one).
3. Split the text into individual grades. Because the numbers are separated by `'\n'`, we can use

`text.split('\n')` to return a **list of strings**, each representing an individual grade. Using the above example, this will return a list that looks like: `['82.5', '93', '77.5', '65']`

4. Calculate and **return the average grade** using this list. Note that this is a list of strings, so to perform arithmetic calculations, you need to cast each string to a `float`.
5. This function should only read from `grades.txt` and should **NOT** write anything to any file.

When you are done writing this function, create a `grades.txt` file yourself and use it to test your function. Specifically, in VSCode, you can click **File -> New Text File**, and write some numbers there, such as the above 4 numbers. **Make sure the file has NO trailing empty line at the end (i.e., your last number should NOT be followed by a newline)** – if you have a trailing empty line, the string's split function will create an empty string at the end of the list, which is not what we want.

Then click **File -> Save As** to save your numbers as `grades.txt` in the **current working directory**. Again, it is generally the directory you see at the Terminal window in VSCode. Finally, in your program, call `print(calc_avg_from_file())` and see if it prints out the correct result. You can also change the numbers in `grades.txt`, save it, and run your program again and verify the result is still correct. If you encounter any problems, use the debugger to help you figure out the issue.

1. Do **NOT** ask the user for any input.
2. Do **NOT** include the function call to this function when you submit. This will fail all the tests in Gradescope.

3. (Common Paragraph) Submission to Gradescope

Log in to [Gradescope](#), select **Lab 9: Files (Code)**, and submit the required file. Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline.

Make sure to select all your group members when submitting. If you see a question you do not know how to answer, ask TAs/UCAs in your lab, use Piazza, or go to one of our office hours.

Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure, it is in your best interest to contact the course staff. We will be using software that will compare your code to other students' in the course, as well as online resources. It is very easy for us to detect similar submissions, which will fail the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the [UMass Academic Honesty Policy and Procedures](#) so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic

dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty, you should re-read the course policies. We assume you have read the course policies in detail, and by submitting this project, you have provided your virtual signature in agreement with these policies.